Paragliding Intelligence Platform

Volume 07 - Mobile & UX Specification v2 Technical

Implementation-ready mobile, web, offline, tracking and UX architecture for a paragliding-first meteo platform

Document status: Draft v2 technical specification

Primary audience: product, mobile, frontend, backend, GIS, weather, safety, AI/ML, QA and investor stakeholders

Revision History

Version	Date	Changes	Owner
v2.0	2026-06-04	Rewritten as technical mobile and UX specification. Adds offline-first architecture, mobile data contracts, background tracking, in-flight mode, design system and QA criteria.	Architecture/Product

Table of Contents

1. Executive Summary

This volume defines the mobile and user-experience architecture for the Paragliding Intelligence Platform. The app is not a generic weather viewer. It is a decision-support, navigation, alerting and coaching interface for paragliding pilots, clubs, schools and competition organizers. The core UX objective is to turn complex forecast, terrain, airspace, live observation and AI data into safe, explainable pilot decisions.

The user experience is built around one operational question: where can I fly, when should I launch, what is the risk, and what changed since the last check? Windy and Ventusky are strong visual weather products, but they do not own the paragliding decision layer. This product should own that layer with site-specific scoring, digital twins, in-flight alerts, AI briefings, pilot-level personalization and offline-safe map/tracking behavior.

Item	Specification
Primary UX promise	Give the pilot a clear GO / MAYBE / NO-GO decision with reasons and time window.
Primary technical promise	Render fast maps, work offline near mountains, track safely, sync reliably and avoid hallucinated safety advice.
Primary differentiator	Decision intelligence plus live reality layer, not only animated weather layers.
Primary safety rule	Never hide severe blockers behind an average score. Storm, gust, rotor, airspace and cloudbase blockers override green UI.

2. Mobile Product Strategy

The mobile app has four modes. Each mode has separate UX density, update cadence and safety responsibility.

Mode	Context	User goal	Core UI
Planning mode	Before travel	Choose launch, compare time windows, read AI briefing, save offline package.	Map, site cards, timeline, route planner
Launch mode	At launch	Validate reality vs forecast and identify last-minute risk change.	Large wind/gust/direction, webcam, pilot reports, blockers
In-flight mode	During flight	Provide minimal, glanceable, audible warnings while preserving battery.	Altitude, airspace, wind drift, landing fields, critical alerts
Debrief mode	After landing	Analyze track, forecast accuracy, climbs, decisions and reports.	IGC import, score, coach, report confirmation

Mobile UX must aggressively reduce cognitive load. A pilot should understand the risk in less than five seconds. Advanced details are available but hidden behind drill-downs.

- A color score is never sufficient alone; every score must have top three reasons.
- No critical alert may require opening a nested menu.
- Offline state must be visible before leaving network coverage.
- The app must distinguish forecast, corrected forecast, live station and pilot report data.
- Every AI recommendation must expose its data inputs and model age.

3. Technology Stack Decision

Recommended architecture: Flutter for iOS/Android mobile, Next.js for web dashboard, MapLibre for map rendering, native location services for tracking, SQLite/Drift or Realm for local storage, and a shared design-token system across mobile and web.

Area	Recommendation	Reason
Mobile framework	Flutter	Fast cross-platform delivery, strong rendering control, mature native bridge for location and offline storage.
Web app	Next.js + TypeScript	SEO landing pages, dashboards, admin, event center and web map.
Map renderer - mobile	MapLibre Native / Flutter MapLibre wrapper	Vector tiles, custom layers, open stack, no lock-in.

Map renderer - web	MapLibre GL JS + deck.gl where needed	WebGL vector tiles, particles, overlays and timeline animation.
Local database	SQLite + Drift/SQLCipher optional	Deterministic offline queries, migration control, encryption option.
State management	Riverpod or Bloc	Predictable state transitions for safety-critical screens.
Background tracking	Native iOS Core Location + Android Fused Location Provider integration	Precise platform behavior and battery control.
Push notifications	Firebase Cloud Messaging + APNs	Critical weather, alert and subscription notifications.

```
mobile_app/
  lib/
    app_shell/
    design_system/
    features/
      map/
      sites/
      flyability/
      alerts/
      tracking/
      airspace/
      ai_briefing/
      xc_planner/
      community/
      billing/
    data/
      api_clients/
      local_db/
      sync/
      repositories/
    native/
      location_bridge/
      background_tasks/
```

4. Information Architecture

The information architecture is tab-based with a persistent safety status layer. It avoids burying core safety information under forecast visualizations.

Navigation item	Scope
Map	Primary landing page; live weather map, launches, timeline, layers, flyability overlay.
Sites	Saved sites, nearby sites, ranked launch recommendations and site search.
Plan	XC route planning, travel radius, route weather, saved forecast packages.
Track	In-flight tracking, IGC recording, airspace warnings, landing fields.
Briefing	AI flight briefing, risk explanation, pilot-specific recommendations.
More	Profile, subscriptions, club/school/event tools, offline maps, settings.

```
RootShell
  - SafetyBanner(global)
  - BottomTabs
      MapTab
      SitesTab
      PlanTab
      TrackTab
      BriefingTab
      MoreTab
  - EmergencyOverlay(priority: critical)
  - OfflineStatusPill
  - EntitlementGate
```

5. Design System

The design system must be deterministic, accessible and safety-aware. Weather apps often rely on beautiful gradients. This platform needs gradients plus categorical safety states that survive sunlight, low battery, color blindness and stress.

State	Meaning	Visual rule	UX rule
GO	Flyable within pilot limits	Green accent plus text label GO	Show reasons and confidence.
MAYBE	Marginal or conditional	Amber accent plus text label MAYBE	Require explicit blockers and validation checklist.
NO-GO	Unsafe or blocked	Red accent plus text label NO-GO	Show blocker first; do not allow score averaging to hide it.
UNKNOWN	Insufficient data	Gray/blue accent plus UNKNOWN	Show missing source, model age and fallback advice.

Token group	Requirement
Typography	Large numbers for wind/gust/cloudbase. Body text no smaller than 13sp on mobile.
Color	Never rely on color alone. All critical states require icon and label.
Spacing	Map overlays use bottom sheets and edge-safe controls for one-handed use.
Haptics	Critical alerts use haptic pulse only when in foreground and user has enabled it.
Voice	In-flight critical warnings can use short audio phrases. No verbose AI speech in flight.

6. App Shell and Navigation

The app shell coordinates authentication state, entitlement state, offline state, active tracking state and safety alerts. It must be independent from feature screens so a critical warning can interrupt any feature.

```
AppShellState {
  authStatus: anonymous | authenticated | expired
  entitlementTier: free | pro | pro_plus | club | school | event | rescue
  networkStatus: online | degraded | offline
  offlinePackageStatus: none | partial | ready | stale
  activeTrackingSessionId?: uuid
  globalSafetyState: go | maybe | no_go | unknown
  unreadCriticalAlerts: int
}
```

State	Navigation behavior
Anonymous user	Can view public map, limited sites and upgrade prompts. No background tracking.
Authenticated free	Can save limited favorites, use basic alerts, no advanced layers.
Pro/Pro+	Unlocks advanced layers, AI briefing, route weather, offline packages.
Club/School/Event	Adds organization dashboards and shared sites/briefings.

7. Map Experience

The map is the cockpit for planning. It must be faster and more decision-oriented than general weather maps. The UX should default to site flyability and allow weather layer drill-down, not the reverse.

Map element	Implementation requirement
Base layer	Vector basemap, terrain shading, contour lines, roads, launch and landing POIs.
Weather layers	Wind particles, wind heatmap, gust, cloudbase, rain, thunderstorm, thermal, airspace, webcams.
Decision overlays	Site flyability circles, route risk line, safe launch window bars, blockers.
Timeline	Horizontal scrubber now to +72h for free/pro, extended range for Pro+.
Altitude selector	Surface, 500m AGL, 1000m AGL, 1500m AGL, 2000m AGL, custom pressure level.

```
MapViewportRequest {
  bbox: [minLon, minLat, maxLon, maxLat]
  zoom: int
  time: iso8601
  layerIds: string[]
  altitudeMode: surface | agl | pressure
  altitudeMeters?: int
  userPilotProfileId?: uuid
}

MapTileResponse {
  vectorTileUrl: string
  rasterOverlayUrl?: string
  particleTileUrl?: string
  expiresAt: iso8601
  attribution: string[]
}
```

7.1 Map Interaction Rules

- Single tap launch: open compact site sheet with current status and next best window.
- Long press map: create temporary point forecast and optional travel target.
- Swipe timeline: update visible flyability, weather overlays and site statuses atomically.
- Layer changes must not reset time or viewport.
- Critical alerts must overlay the map without hiding own-position marker.

8. Site Discovery and Ranking

Site discovery ranks flying sites by paragliding fit, not generic weather quality. Ranking includes drive time, pilot skill, site orientation, actual observations, model disagreement and subscription tier.

```
GET /v1/mobile/sites/recommendations?lat=61.88&lon=9.1&radius_km=180&pilot_level=intermediate&time=2026-06-04T12:00:00Z

SiteRecommendation {
  siteId: uuid
  name: string
  distanceKm: number
  driveTimeMinutes?: int
  status: GO | MAYBE | NO_GO | UNKNOWN
  flyabilityScore: int
  safetyScore: int
  xcScore?: int
  bestWindow: { start: iso8601, end: iso8601 }
  topReasons: string[]
  blockers: string[]
}
```

Priority	Rule
Rank 1	NO-GO blockers always push site below GO/MAYBE sites unless user filters explicitly for analysis.
Rank 2	Prefer higher confidence over higher raw flyability when model disagreement is high.
Rank 3	Respect pilot profile and wing class; beginner results should be conservative.
Rank 4	Include travel time only as a secondary score. A far safer site beats a nearby marginal site.

9. Launch Detail Screen

Launch detail is the primary decision screen. It must combine current status, time windows, reasons, live reality and local hazards.

Section	Required content
Header	Site name, country/region, difficulty, current GO/MAYBE/NO-GO.
Decision card	Flyability score, safety score, XC score, confidence, last update time.
Best window	Hour-by-hour status with launch window recommendation.
Reality check	Nearest stations, webcams, pilot reports, model vs reality delta.
Hazards	Rotor zones, power lines, landing notes, local rules, airspace.
Actions	Navigate, save offline, set alert, start tracking, ask AI, share briefing.

```
LaunchDetailViewModel {
  site: SiteSummary
  currentDecision: FlyabilityDecision
  hourly: FlyabilityHour[]
  liveReality: RealitySnapshot
  hazards: Hazard[]
  webcams: WebcamStatus[]
  airspace: AirspaceSummary
  offlineAvailability: available | pro_required | unavailable
}
```

10. Flyability Timeline

The timeline converts weather complexity into actionable windows. It must expose both score and blocker transitions.

Component	Requirement
Hourly row	Time, status, wind, gust, direction, cloudbase, thermal, rain/storm, confidence.
Collapsed mobile view	Colored bars plus top blocker icon. Tap expands variables.
Comparison mode	Forecast model A/B differences for Pro+.
Reality overlay	Station and pilot report markers on current hour.

```
FlyabilityHour {
  validTime: iso8601
  status: GO | MAYBE | NO_GO | UNKNOWN
  flyabilityScore: int
  safetyScore: int
  confidence: number
  windKmh: number
  gustKmh: number
  windDirectionDeg: int
  cloudbaseMslM: int
  thermalStrengthMs?: number
  blockers: Blocker[]
  explanation: string[]
}
```

11. Weather Layers UX

Weather layers must support both exploratory meteorology and fast launch decisions. Advanced layers are grouped by pilot task, not by meteorological taxonomy.

Layer group	Layers
Launch safety	Surface wind, gust, direction, rotor, valley wind, foehn, webcam, live station.
XC potential	Thermal strength, cloudbase, cloud streets, wind at altitude, expected climb.
Storm avoidance	Rain, radar, lightning, CAPE/instability, overdevelopment risk.

Navigation	Airspace, landing fields, terrain, glide reach, obstacles.

Tier	Layer entitlements
Free	Basic wind, gust, rain, launch markers, 3-day forecast.
Pro	Thermals, cloudbase, flyability timeline, unlimited favorites, offline basic packages.
Pro+	Model comparison, corrected forecast, route weather, AI briefing, advanced offline.
Club/School/Event	Shared overlays, member tracking, task overlays, organization alerts.

12. AI Flight Briefing UX

AI is a briefing interface, not a weather oracle. The UX must show that every AI statement comes from retrieved forecast, site, pilot and live data. The AI cannot override hard safety blockers.

```
AIBriefingRequest {
  question: string
  userLocation?: { lat: number, lon: number }
  siteId?: uuid
  timeRange: { start: iso8601, end: iso8601 }
  pilotProfileId: uuid
  includeSources: true
}

AIBriefingResponse {
  answer: string
  recommendation: GO | MAYBE | NO_GO | UNKNOWN
  confidence: number
  groundedFacts: GroundedFact[]
  blockers: Blocker[]
  followUpActions: Action[]
  generatedAt: iso8601
}
```

UX element	Requirement
Grounding panel	Expandable list of data facts used by the AI, including forecast model and age.
Safety footer	Always visible disclaimer: verify actual conditions at launch.
Action buttons	Save briefing, share, set alert, compare sites, download offline package.
Hallucination control	If data missing, AI must say UNKNOWN and request/offer concrete missing source.

13. Alert Center

Alerts are rule-based and safety-prioritized. The UI separates opportunity alerts from danger alerts.

Alert class	Examples
Opportunity alert	Site becomes GO, cloudbase exceeds threshold, XC score improves, wind aligns.
Warning alert	Gust limit exceeded, storm risk increased, model confidence dropped, airspace change.
Critical alert	Lightning nearby, severe gust increase, airspace violation imminent, emergency tracking event.

```
AlertRule {
  id: uuid
  userId: uuid
  scope: site | area | route | organization
  condition: jsonlogic
  severity: opportunity | warning | critical
```

```
  channels: push | email | sms | in_app
  quietHours?: { startLocal: string, endLocal: string }
  ttlMinutes: int
}
```

14. Live Tracking and In-flight Mode

In-flight mode prioritizes readability, battery life and interrupt-driven warnings. The interface must be usable with gloves, sunlight and turbulence.

Area	Requirement
Primary metrics	Altitude, ground speed, vario if connected, wind drift, glide reach, airspace distance.
Controls	Start/stop track, lock screen, mark thermal, report condition, SOS/share live link.
Map overlays	Own position, route, airspace, landing fields, hazard sectors, nearest safe landing.
Audio	Short warnings only: Airspace ahead, land soon, storm risk, tracking lost.
Battery modes	High precision, balanced, battery saver; automatically suggest saver under 25 percent.

```
TrackingSample {
  sessionId: uuid
  recordedAt: iso8601
  lat: number
  lon: number
  gpsAltitudeM?: number
  baroAltitudeM?: number
  groundSpeedKmh?: number
  headingDeg?: int
  verticalSpeedMs?: number
  batteryPct?: int
  accuracyM?: number
}

WebSocket /v1/ws/tracking/{sessionId}
  client -> server: TrackingSampleBatch
  server -> client: InFlightAlert | Ack | RouteWeatherUpdate
```

15. Airspace Guardian UX

The airspace guardian must work offline for downloaded regions. It converts complex airspace geometry into distance, time-to-entry and vertical conflict warnings.

Mode	Requirement
Pre-flight	Show airspace near launch and along planned XC route.
In-flight passive	Display nearest controlled/restricted airspace and ceiling/floor.
In-flight active	Warn when projected track intersects restricted airspace.
Post-flight	Flag possible airspace proximity events in debrief.

```
AirspaceWarning {
  airspaceId: uuid
  classCode: string
  name: string
  horizontalDistanceM: int
  verticalMarginM?: int
  projectedEntrySeconds?: int
  severity: info | warning | critical
  instruction: string
}
```

16. XC Planner UX

XC planner helps a pilot choose route candidates based on thermals, cloudbase, wind drift, landing fields and airspace. It must not pretend to guarantee distance.

Component	Requirement
Route cards	20 km easy, 50 km standard, 100 km ambitious, custom route.
Route weather	Hourly segment risk, altitude wind, thermal probability, landing options.
Task tools	Turnpoints, cylinders, upload/download task files in later versions.
Failure handling	If confidence low, route remains visible but warning appears before export.

```
XCPlanRequest {
  launchSiteId: uuid
  startWindow: { start: iso8601, end: iso8601 }
  targetDistanceKm?: int
  pilotProfileId: uuid
  avoidAirspace: true
  requireLandingFields: true
}

XCPlanResponse {
  candidateRoutes: XCRoute[]
  confidence: number
  mainRisks: string[]
}
```

17. Community and Reality Layer

The reality layer lets pilots compare forecast against actual launch and flight conditions. It is a strategic differentiator because it creates data competitors cannot easily copy.

Reality input	UX/data requirement
Pilot report	Wind, gust, direction, cloudbase, thermal, launch crowding, safety note, photo optional.
Station reading	Automated wind/temperature/humidity with source reliability.
Webcam observation	Computer-vision derived cloud/visibility/windsock state.
Trust score	Reports weighted by pilot reputation, recency, proximity and consistency.

```
PilotObservationDraft {
  siteId?: uuid
  lat: number
  lon: number
  observedAt: iso8601
  windKmh?: number
  gustKmh?: number
  windDirectionDeg?: int
  cloudbaseMslM?: int
  thermalStrengthMs?: number
  note?: string
  visibility: public | club | private
}
```

18. Digital Twin Insights

Digital twin output should be visible but understandable. The user should see local correction and confidence, not ML jargon.

Insight	UX requirement
Forecast correction	Show: model says 14 km/h, local correction predicts 21 km/h.
Historical pattern	Show: this launch often overperforms in NW after 12:00.

Risk memory	Show common rotor/valley wind pattern under current direction.
Confidence	Show enough history / limited history / conflicting reports.

19. Club, School and Event UX

Module	Mobile/web requirement
Club dashboard	Shared forecast board, member check-ins, site report moderation, local alerts.
School dashboard	Student skill matrix, safe training windows, instructor briefing, attendance.
Event dashboard	Task map, weather center, pilot tracking, safety notices, organizer override.
Rescue view	Last known positions, drift estimate, weather reconstruction, search notes.

```
OrganizationContext {
  organizationId: uuid
  role: owner | admin | instructor | safety_officer | member | student | event_staff
  permissions: string[]
  activeEventId?: uuid
}
```

20. Account, Billing and Entitlements

Entitlements must be explicit, cached and fail-safe. A paid feature must either work offline because its data is already downloaded or clearly explain that network/subscription validation is required.

Tier	Mobile entitlement behavior
Free	Basic forecast, limited favorites, basic alerts, limited forecast range.
Pro	Flyability timeline, thermals/cloudbase, unlimited favorites, basic offline.
Pro+	Model comparison, corrected forecast, AI briefing, XC planner, advanced offline.
Club/School/Event	Organization features, shared dashboards, event/task features.

```
EntitlementSnapshot {
  userId: uuid
  tier: free | pro | pro_plus | club | school | event | rescue
  validUntil: iso8601
  features: string[]
  offlineGraceUntil?: iso8601
  source: server | cached
}
```

21. Offline-first Mobile Architecture

Paragliding often happens in mountains with weak connectivity. Offline architecture is not an extra; it is core safety infrastructure.

Offline element	Requirement
Offline package	Site data, base map tiles, terrain tiles, airspace, latest forecast, hazard notes, alert rules.
Package scope	By site, route corridor, club region or event task area.
Freshness rules	Forecast package has visible expiry; stale forecasts cannot show GO state.
Conflict policy	Local pilot reports sync later with server conflict resolution.

```
OfflinePackage {
  id: uuid
  type: site | route | region | event
  bbox: [number, number, number, number]
```

```
    siteIds: uuid[]
    validFrom: iso8601
    validUntil: iso8601
    includes: [basemap, terrain, airspace, forecast, hazards, alerts]
    bytes: int
    status: downloading | ready | stale | expired | failed
  }
```

22. Local Storage and Sync

Local storage should use a clear repository layer. Screens must not directly query APIs. They read repositories that merge local cache, pending writes and server snapshots.

Local table	Purpose
local_sites	Site summary, hazards, directions, offline package membership.
local_forecast_hours	Site forecast hours and derived flyability.
local_airspace_tiles	Airspace vector tile metadata and expiry.
local_tracks	Tracking sessions and unsynced samples.
outbox	Pending observations, track uploads, alert changes.
sync_state	Last sync, error, retry-after, schema version.

```
Sync policy:
1. Reads prefer fresh local cache for fast UI.
2. Network refresh runs in background if connectivity exists.
3. User writes go to local outbox first.
4. Outbox retries with exponential backoff and idempotency keys.
5. Critical alerts are fetched via push and confirmed by API when online.
6. Stale forecast data cannot produce GO; it degrades to UNKNOWN or MAYBE depending on blocker state.
```

23. Background Location, Battery and Permissions

Background location must be treated as sensitive and safety-critical. The app only requests background location when the user enables live tracking, event tracking or rescue sharing. It must explain the purpose clearly and provide stop controls.

Platform/area	Requirement
iOS	Use Core Location background updates only for explicit tracking mode. Show system-compliant purpose copy.
Android	Foreground service for active in-flight tracking, notification visible, background access only if justified.
Battery	Adaptive sample rate based on speed, altitude change, airspace proximity and battery level.
Privacy	Live location sharing defaults to private; public sharing requires explicit opt-in.

```
Tracking sample rate policy:
- Ground / stationary: 60-180 seconds or motion-triggered.
- Hiking to launch: 15-60 seconds.
- In flight normal: 2-5 seconds.
- Near airspace / critical alert: 1-2 seconds temporarily.
- Battery < 15 percent: degrade to 10-15 seconds unless critical alert active.
```

24. Accessibility and Internationalization

Area	Requirement
Color blindness	Status labels and icons required in addition to color.
Sunlight	High contrast in-flight theme.
Gloves	Large touch targets for in-flight controls.
Screen readers	Map alternatives: site list, current status, alerts and timeline readouts.
Language	All text externalized; weather units localized; aviation terms kept consistent.
Units	km/h, m/s, knots, meters, feet, Celsius/Fahrenheit configurable.

25. Performance Budgets

Metric	Budget
Cold start to map skeleton	< 2.5 seconds on mid-range device.
Open saved launch detail offline	< 500 ms.
Map pan frame rate	Target 45-60 FPS with one animated layer; degrade gracefully.
Timeline scrub response	< 150 ms UI response using cached tiles.
Track sample write	< 50 ms local write.
Battery drain in tracking	Target less than 8-12 percent/hour in balanced mode, device dependent.
Offline package download resume	Must resume after connection loss.

26. Analytics and Telemetry

Analytics must improve product quality without turning safety decisions into manipulative engagement design. The app should track decision quality, forecast freshness, conversion and retention, but avoid dark patterns.

Event	Purpose
map_layer_enabled	Layer usage and performance.
site_recommendation_opened	Recommendation effectiveness.
flyability_decision_viewed	Decision feature engagement.
alert_created	Alert demand and tier conversion.
offline_package_downloaded	Safety preparedness.
tracking_started/stopped	Flight activity and battery mode.
ai_briefing_generated	AI feature usage and grounding.
no_go_blocker_seen	Safety UX visibility.

```
AnalyticsEvent {
  eventName: string
  userIdHash?: string
  sessionId: uuid
  timestamp: iso8601
  properties: object
  privacyClass: product | safety | billing | diagnostic
}
```

27. Error States and Safety Copy

State	Required copy behavior
Forecast stale	Forecast is older than allowed. Do not use for launch decision.
Model disagreement high	Models disagree strongly. Treat as MAYBE and verify at launch.
Station offline	Nearest station has not updated. Use forecast only with caution.
GPS degraded	Location accuracy low. Airspace warnings may be delayed.
Offline expired	Offline weather package expired. Map and airspace remain available; forecast is disabled.
AI unavailable	AI briefing unavailable. Use structured forecast and safety blockers.

28. API Contracts Used by Mobile

Mobile consumes a curated API layer. It should not call internal weather cube services directly.

Endpoint	Mobile use
GET /v1/mobile/bootstrap	Initial config, feature flags, entitlements, app min version.
GET /v1/sites/recommendations	Ranked launch recommendations for current profile and time.
GET /v1/sites/{id}/mobile-detail	Launch detail data optimized for mobile.
GET /v1/flyability/site/{id}	Hourly flyability timeline.
POST /v1/briefings	Grounded AI flight briefing.
POST /v1/offline-packages	Create/download offline package manifest.

POST /v1/tracking/sessions	Start tracking session.
WS /v1/ws/tracking/{sessionId}	Live tracking samples and warnings.
POST /v1/observations	Pilot report submission.
GET /v1/airspace/warnings	Projected airspace conflict check.

29. QA Acceptance Criteria

Test area	Acceptance criterion
Map launch status	Given a site with hard storm blocker, UI shows NO-GO regardless of average flyability score.
Offline package	Given downloaded site package and no network, launch detail opens and displays expiry state.
Tracking loss	Given network loss during active tracking, samples store locally and sync later with no duplicate track.
Airspace alert	Given projected route intersects restricted airspace, warning appears within configured threshold.
AI grounding	Given missing weather data, AI returns UNKNOWN and does not invent wind/cloudbase.
Entitlement gate	Given free user opens Pro+ layer, UI explains benefit and shows upgrade without breaking map.
Accessibility	Given color-blind mode or screen reader, GO/MAYBE/NO-GO state remains understandable.

30. MVP and Roadmap

Phase	Mobile/UX scope
MVP	Map, site discovery, launch detail, flyability timeline, basic alerts, profile, limited offline, basic tracking.
V1	Thermals/cloudbase layers, AI briefing, Pro/Pro+ billing, community reports, route weather.
V2	Advanced offline, airspace guardian, digital twin insights, live reality layer, webcam ingestion.
V3	Competition mode, club/school dashboards, AI debrief, rescue tools, hardware integrations.

Backlog ID	Engineering task
MOB-001	Build app shell, bottom navigation and global safety banner.
MOB-002	Implement MapLibre vector basemap and launch markers.
MOB-003	Implement launch detail view model and local cache.
MOB-004	Implement flyability timeline component with blockers.
MOB-005	Implement offline package manifest and local persistence.
MOB-006	Implement tracking session with local sample store and batch upload.
MOB-007	Implement AI briefing screen with grounded facts panel.
MOB-008	Implement alert center and push notification handling.

31. Source References

These references informed technology choices and mobile/platform constraints. They should be reviewed during implementation because mobile OS rules, SDK behavior and store policies can change.

Reference	URL
MapLibre GL JS documentation	https://www.maplibre.org/maplibre-gl-js/docs/
MapLibre main site	https://maplibre.org/
MapLibre plugins/offline references	https://www.maplibre.org/maplibre-gl-js/docs/plugins/
Android location permissions	https://developer.android.com/develop/sensors-and-location/location/permissions
Android background location access	https://developer.android.com/develop/sensors-and-location/location/background
Apple Core Location background updates	https://developer.apple.com/documentation/corelocation/handling-location-updates-in-the-background
Apple Core Location documentation	https://developer.apple.com/documentation/corelocation

Flutter geolocator package	https://pub.dev/packages/geolocator
Flutter background geolocation package reference	https://pub.dev/packages/flutter_background_geolocation

Appendix A - Compact Screen Inventory

ID	Screen	Purpose
S-001	Onboarding	Profile, skill, wing class, units, privacy explanation.
S-002	Home Map	Map, timeline, layer picker, launch markers.
S-003	Site Detail	Decision card, timeline, reality, hazards, actions.
S-004	Site Search	Nearby, favorites, filters, map/list toggle.
S-005	AI Briefing	Chat, answer, sources, actions, safety footer.
S-006	Alert Rules	Create/edit conditions, scope, channels, severity.
S-007	Offline Packages	Downloaded regions, expiry, storage, update.
S-008	Tracking	In-flight map, metrics, airspace, SOS/share.
S-009	XC Planner	Route candidates, risk segments, export/share.
S-010	Community Report	Observation form, photo, visibility, trust.
S-011	Subscription	Tier comparison, paywall, restore purchases.
S-012	Organization	Club/school/event dashboard entry point.

Appendix B - Mobile Repository Interfaces

```
abstract class SiteRepository {
  Future<List<SiteRecommendation>> getRecommendations(SiteQuery query);
  Future<LaunchDetailViewModel> getLaunchDetail(String siteId, {bool preferCache = true});
  Stream<LaunchDetailViewModel> watchLaunchDetail(String siteId);
}

abstract class TrackingRepository {
  Future<TrackingSession> startSession(TrackingStartRequest request);
  Future<void> appendSamples(List<TrackingSample> samples);
  Stream<InFlightAlert> watchAlerts(String sessionId);
  Future<void> stopSession(String sessionId);
}

abstract class OfflinePackageRepository {
  Future<OfflinePackageManifest> createManifest(OfflinePackageRequest request);
  Stream<OfflinePackageProgress> download(String packageId);
  Future<bool> isReadyForSite(String siteId);
}
```